Logging: A Practical Deep Dive

A hands-on guide for backend developers (with Spring Boot examples)

What you will learn

• What logs are and why they matter in production

• Log levels (TRACE/DEBUG/INFO/WARN/ERROR) and how to choose them

• Where to add logs: controller, service, DB, external calls, exceptions

• Structured logging (JSON), correlation IDs and trace IDs

• Audit logs vs application logs

• Best practices + common mistakes

• Spring Boot configuration + examples

• How logs reach files and dashboards (ELK/Splunk/Cloud)

1. Logging Basics

Logging is the practice of recording events from a running application. Think of it as the system's diary: it captures what happened, when it happened, and (often) who triggered it.

Why logging exists

• Debug production issues where debugger access is impossible

• Understand system behavior: slow APIs, retries, failures

• Security investigations and compliance (especially with audit logs)

• Operational visibility: traffic spikes, errors, dependency health

Typical log line

```
2026-01-18 16:30:10.123 INFO OrderService - Order created | orderId=O123 | userId=U88 | latencyMs=84
```

2. Where Logs Are Added

A simple rule: log important state changes, decisions, and failures. Avoid logging every line.

Controllers (entry/exit)

• Incoming request started

• Key request parameters (non-sensitive)

• Response status + latency

Service layer (business decisions)

• Branch decisions (e.g., coupon applied)

• Important domain events (order created)

• Retries/circuit-breaker events

External calls (HTTP/DB/Queue)

• Before calling dependency

• After call: status + latency

• Timeouts and fallbacks

Exception boundary

• Unexpected errors with stack trace

• Handled errors with reason code

3. Log Levels: Choosing the Right One

Level	When to use	Example
TRACE	Very detailed, rarely enabled	Entering/exiting methods
DEBUG	Developer details for debugging	Computed totals, flags
INFO	Normal business events	Order created, user login
WARN	Something unexpected but app continues	Slow call, retry happened
ERROR	Request failed / incident	Unhandled exception, dependency down

Golden rule: If it will help you answer “What happened?” at 2 AM during an incident, it belongs at INFO/WARN/ERROR. Debug/Trace are for development.

4. When Do Logs Print?

A log line is printed when two conditions are true:

• The code executes that log statement (the line is reached)

• The log level is enabled for that package/class

Example: if your app runs at INFO level, DEBUG logs will not appear.

5. Console vs File vs Centralized Logging

Locally, logs usually print to the console. In production, logs are shipped to files and then to centralized systems.

Environment	Common destination	Why
Local dev	Console (IDE/Terminal)	Fast feedback
Staging	Files + dashboard	Reproduce incidents safely
Production	Centralized logging (ELK/Splunk/Cloud)	Search, alerts, retention

Even if you see logs in the console, most real systems also write them to rotating files and forward them to a log platform.

6. Correlation IDs (Request ID) - Make Logs Debuggable

In microservices, one user request touches multiple services. To connect those logs together, we attach a correlation ID (request ID) and propagate it across calls.

Example idea:

• Gateway generates requestId=R1
• Order service logs [requestId=R1]
• Payment service logs [requestId=R1]

This single trick reduces debugging time dramatically.

7. Application Logs vs Audit Logs

Not all logs are the same. Application logs help operations; audit logs help security and compliance.

Type	Purpose	Example events	Should be editable?
Application logs	Debug + monitor app health	timeouts, retries, slow queries	Yes (normal retention/rotation)
Audit logs	Proof of user/admin actions	login, role change, money transfer	No (immutable / append-only)

Audit logs are often stored in a separate secure store and protected against tampering.

8. Structured Logging (JSON)

Plain text logs are hard to search. Structured logs store fields explicitly so tools can filter and aggregate them.

Example JSON log:

```
{"ts":"2026-01-18T16:30:10.123+05:30","level":"INFO","service":"order","msg":"order_created
","orderId":"O123","userId":"U88","latencyMs":84}
```

9. Spring Boot: Practical Setup

Spring Boot uses SLF4J API with Logback by default.

Add a logger in any class:

```
private static final Logger log = LoggerFactory.getLogger(OrderService.class);
```

Common patterns:

```
log.info("Order created | orderId={} | userId={}", orderId, userId);
```

Log exceptions properly:

```
try { ... } catch(Exception e) { log.error("Payment failed", e); }
```

Enable file logging (example):

```
logging.file.name=logs/app.log logging.level.root=INFO
```

Note: configure rotation in logback-spring.xml for production.

10. Best Practices (Do this / Avoid this)

Do

• Log with context: ids, status, latency

• Use correlation IDs for request tracking

• Prefer structured logs for production

• Log at boundaries: entry/exit, external calls, exceptions

• Redact sensitive data (passwords, OTP, full cards)

Avoid

• Don't log secrets: passwords, tokens, private keys

• Don't spam logs inside tight loops

• Don't use ERROR for expected validation failures

• Don't log full payloads blindly

• Don't rely on console logs in production

Quick checklist: Can I answer these from logs? 1) What happened? 2) For which user/order? 3) Where did it fail? 4) How long did it take?